
Computer System Performance

JAC Hermocilla

September 15, 2025

Computer System Performance Tutorial

Table of Contents

1. Introduction to Performance
2. Performance Metrics
3. Components of Execution Time
4. Speedup and Performance Improvement
5. Instructions Per Second
6. Benchmarks
7. Worked Examples
8. Advanced Topics
9. Summary

Introduction to Performance

Computer system performance is a critical aspect of computer architecture that determines how efficiently a system can execute programs. Performance analysis helps us understand bottlenecks, compare different systems, and make informed design decisions.

Key Performance Questions

- How fast does my program run on this machine?
- Which machine is faster for my application?
- How much improvement will I get from a new processor?
- What limits the performance of my system?

Performance Metrics

Response Time vs Throughput

Response Time (Execution Time): The total time to complete a single task

- Also called latency
- Measured from start to completion of a program
- Critical for individual user experience

Throughput: The number of tasks completed per unit time

- Also called bandwidth
- Important for server applications
- Can be improved without improving response time

Performance Definition

Performance is inversely related to execution time:

$$\text{Performance} = 1 / \text{Execution Time}$$

thus

$$\text{Execution Time} = 1 / \text{Performance}$$

If machine A runs a program in 10 seconds and machine B runs it in 15 seconds:

- $\text{Performance}_A = 1/10 = 0.1$ tasks/second
- $\text{Performance}_B = 1/15 = 0.0666$ tasks/second
- Machine A is (1.5 times) faster than machine B

Components of Execution Time

The fundamental performance equation breaks down CPU execution time into three key components:

The Iron Law of Performance

$$\text{CPU Execution Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

Where:

- **Instruction Count (IC):** Total number of instructions executed
- **CPI:** Clock cycles Per Instruction (average)
- **Clock Cycle Time:** Time for one clock cycle (1/Clock Rate)

Alternative Formulation

$$\text{CPU Execution Time} = (\text{Instruction Count} \times \text{CPI}) / \text{Clock Rate}$$

Understanding Each Component

1. Instruction Count (IC)

- Determined by the program, compiler, and instruction set architecture (ISA)
- Different programs have different instruction counts
- Compiler optimizations can reduce instruction count
- ISA design affects how many instructions are needed

2. Clock Cycles Per Instruction (CPI)

- Average number of clock cycles each instruction takes
- Depends on processor implementation and instruction mix
- Simple instructions (add, move): 1-2 cycles
- Complex instructions (divide, memory access): 10+ cycles
- Pipeline efficiency affects CPI

3. Clock Rate (Frequency)

- Number of clock cycles per second (Hz)
- Higher clock rate = shorter clock cycle time
- Limited by circuit delays and power consumption
- Modern processors: 1-5 GHz

Factors Affecting Each Component

Component	Affected By
Instruction Count	Algorithm, Programming Language, Compiler, ISA
CPI	ISA, Processor Implementation, Cache Performance
Clock Rate	Circuit Technology, Processor Organization

Speedup and Performance Improvement

Speedup Definition

Speedup measures how much faster one system is compared to another:

$$\text{Speedup} = \text{Performance}_{\text{new}} / \text{Performance}_{\text{old}} = \text{Time}_{\text{old}} / \text{Time}_{\text{new}}$$

Percent Speedup

$$\text{Percent Speedup} = ((\text{Time_old} - \text{Time_new}) / \text{Time_old}) \times 100\%$$

Or equivalently:

$$\text{Percent Speedup} = ((\text{Speedup} - 1) / 1) \times 100\%$$

Amdahl's Law

Amdahl's Law quantifies the maximum speedup achievable when only part of a system is improved:

$$\text{Speedup_overall} = 1 / ((1 - P) + P/S)$$

Where:

- P = Fraction of execution time affected by improvement
- S = Speedup of the affected portion

Key Insight: The performance improvement is limited by the portion that cannot be improved.

Instructions Per Second**MIPS (Million Instructions Per Second)**

$$\text{MIPS} = \text{Instruction Count} / (\text{Execution Time} \times 10^6)$$

Or:

$$\text{MIPS} = \text{Clock Rate} / (\text{CPI} \times 10^6)$$

FLOPS (Floating Point Operations Per Second)

For floating-point intensive applications:

$$\text{FLOPS} = \text{Number of FP Operations} / \text{Execution Time}$$

Common units: MFLOPS, GFLOPS, TFLOPS

Limitations of MIPS and MFLOPS

- Don't account for instruction complexity differences
- Can be misleading when comparing different ISAs
- Don't reflect actual program performance

Benchmarks

Types of Benchmarks

1. Microbenchmarks

- Test specific processor features
- Examples: Memory bandwidth, cache latency, floating-point performance

2. Kernel Benchmarks

- Small, key pieces of real applications
- Examples: Matrix multiply, FFT, sorting algorithms

3. Application Benchmarks

- Real applications or application suites
- Examples: SPEC CPU, TPC benchmarks, PARSEC

4. Synthetic Benchmarks

- Artificial programs designed to match workload characteristics
- Examples: Dhrystone, Whetstone

SPEC (Standard Performance Evaluation Corporation)

SPEC CPU benchmarks are industry-standard application benchmarks:

- **SPEC CPU**: Tests processor and memory performance
- **SPECint**: Integer-intensive programs
- **SPECfp**: Floating-point intensive programs

Results reported as ratios compared to reference machine.

Worked Examples

Example 1: Basic Performance Calculation

Problem: A program executes 2 billion instructions on a 3 GHz processor with an average CPI of 1.5. Calculate the execution time.

Solution:

Given:

- Instruction Count = 2×10^9
- Clock Rate = 3×10^9 Hz
- CPI = 1.5

CPU Execution Time = $(IC \times CPI) / \text{Clock Rate}$

CPU Execution Time = $(2 \times 10^9 \times 1.5) / (3 \times 10^9)$

CPU Execution Time = $3 \times 10^9 / 3 \times 10^9 = 1$ second

Example 2: Comparing Two Processors

Problem: Compare two processors running the same program:

- Processor A: 4 GHz, CPI = 2.0
- Processor B: 3 GHz, CPI = 1.2
- Instruction count is the same for both

Solution:

For same instruction count, we can compare execution times:

Time_A approx= $CPI_A / \text{Clock_Rate_A} = 2.0 / 4 = 0.5$

Time_B approx= $CPI_B / \text{Clock_Rate_B} = 1.2 / 3 = 0.4$

Speedup_B_over_A = $\text{Time_A} / \text{Time_B} = 0.5 / 0.4 = 1.25$

Processor B is 1.25× faster (25% speedup)

Example 3: Amdahl's Law Application

Problem: A program spends 60% of its time in floating-point operations. We upgrade the floating-point unit to be 4× faster. What's the overall speedup?

Solution:

Given:

- $P = 0.6$ (60% in floating-point)
- $S = 4$ (4× speedup for FP operations)

$$\text{Speedup_overall} = 1 / ((1 - P) + P/S)$$

$$\text{Speedup_overall} = 1 / ((1 - 0.6) + 0.6/4)$$

$$\text{Speedup_overall} = 1 / (0.4 + 0.15)$$

$$\text{Speedup_overall} = 1 / 0.55 = 1.82$$

Overall speedup is 1.82× (82% improvement)

Example 4: MIPS Calculation

Problem: A processor runs at 2.5 GHz with an average CPI of 2.2. Calculate its MIPS rating.

Solution:

Given:

- Clock Rate = 2.5×10^9 Hz
- CPI = 2.2

$$\text{MIPS} = \text{Clock Rate} / (\text{CPI} \times 10^6)$$

$$\text{MIPS} = (2.5 \times 10^9) / (2.2 \times 10^6)$$

$$\text{MIPS} = 2500 / 2.2 = 1136 \text{ MIPS}$$

Example 5: Performance Improvement Analysis

Problem: We're considering three improvements to a processor:

1. Reduce CPI from 2.0 to 1.6 (20% reduction)
2. Increase clock rate from 3 GHz to 3.6 GHz (20% increase)
3. Reduce instruction count by 15% through compiler optimization

Calculate the individual and combined speedups.

Solution:

Original: IC_0 , $CPI_0 = 2.0$, $Clock_0 = 3 \text{ GHz}$

Individual Speedups:

1. CPI improvement: $Speedup_1 = CPI_0 / CPI_1 = 2.0 / 1.6 = 1.25$
2. Clock improvement: $Speedup_2 = Clock_1 / Clock_0 = 3.6 / 3.0 = 1.2$
3. IC improvement: $Speedup_3 = IC_0 / IC_1 = 1 / 0.85 = 1.176$

Combined Speedup:

All three improvements multiply:

$$Speedup_{total} = 1.25 \times 1.2 \times 1.176 = 1.764$$

The combined improvement gives 76.4% speedup.

Example 6: Benchmark Comparison

Problem: Two machines run a benchmark suite with the following results:

Benchmark	Machine A (seconds)	Machine B (seconds)
Prog1	10	8
Prog2	20	25
Prog3	5	4
Prog4	15	12

Calculate arithmetic and harmonic mean speedups.

Solution:

Individual Speedups:

$$\text{Prog1: } 10/8 = 1.25$$

$$\text{Prog2: } 20/25 = 0.8$$

$$\text{Prog3: } 5/4 = 1.25$$

$$\text{Prog4: } 15/12 = 1.25$$

Arithmetic Mean Speedup:

$$AM = (1.25 + 0.8 + 1.25 + 1.25) / 4 = 1.14$$

For Harmonic Mean, use execution times:

Total time A = $10+20+5+15 = 50$ seconds

Total time B = $8+25+4+12 = 49$ seconds

Harmonic Mean Speedup = $50/49 = 1.02$

Machine A is slightly faster overall, but the difference is small.

Advanced Topics

Performance Counters

Modern processors include hardware performance counters that measure:

- Instructions retired
- Cache hits/misses
- Branch mispredictions
- Pipeline stalls

These provide detailed insight into performance bottlenecks.

Power and Energy Considerations

Performance isn't just about speed anymore:

$$\text{Energy} = \text{Power} \times \text{Time}$$

Sometimes reducing performance can save more energy than the time penalty costs.

Multicore Performance

For parallel applications:

- **Parallel Efficiency:** Speedup / Number of Cores
- **Scalability:** How performance changes with core count
- **Load Balancing:** Even distribution of work

Memory System Impact

Modern systems are often memory-bound:

- **Memory Wall:** Gap between processor and memory speed
- **Cache Performance:** Critical for overall system performance
- **Memory Bandwidth:** Limits throughput for data-intensive applications

Summary

Key Takeaways

1. **Performance = 1/Execution Time** - Lower execution time means higher performance
2. **Iron Law:** $\text{CPU Time} = \text{IC} \times \text{CPI} \times \text{Clock Cycle Time}$ - All three factors matter
3. **Optimization Trade-offs:** Improving one factor may worsen others
4. **Amdahl's Law:** Overall speedup is limited by the unimproved portion
5. **Benchmarks:** Use representative workloads for meaningful comparisons
6. **Context Matters:** Performance metrics should match your application needs

Performance Optimization Strategy

1. **Measure First:** Use profiling to find bottlenecks
2. **Target Bottlenecks:** Focus on the most significant performance limiters
3. **Consider All Factors:** Don't optimize in isolation
4. **Validate Improvements:** Measure actual performance gains
5. **Balance Trade-offs:** Consider power, cost, and complexity

Common Pitfalls

- Comparing systems using only one metric (e.g., clock rate)
- Ignoring the workload when evaluating performance
- Optimizing non-critical code paths
- Assuming synthetic benchmarks represent real applications
- Not considering system-level effects (OS overhead, I/O, etc.)

Acknowledgement

Help provided by ClaudeAI (accessed 2025-09-15)